Informe final — Fase II

Centro Médico Pediátrico · MongoDB

Bases de Datos II · Proyecto final · Integrante 2

Universidad Rafael Landívar — Facultad de Ingeniería

24 de octubre de 2026 · Versión que reemplaza al informe del 23 de octubre

Estado final del módulo individual: auditoría del código frente a los requisitos, defectos encontrados y corregidos, verificación con MongoDB real y navegador real, limitaciones propias de MongoDB y pendientes de integración con el equipo. Todos los datos son ficticios y el sistema no genera indicaciones clínicas.

1. Resumen ejecutivo

El módulo pediátrico está completo dentro de la responsabilidad individual: la aplicación inicia sin errores, permite operar con los tres roles de desarrollo y persiste correctamente en MongoDB. Quedan **dos acciones que requieren privilegios de administrador del servidor** (aplicar los validadores de Fase II y, de forma opcional, migrar a replica set) y las tareas de integración con el equipo. Ninguna se presenta como terminada.

Indicador	Resultado verificado
npm test	61 pruebas: 61 aprobadas, 0 fallidas, 0 omitidas (antes: 44 pruebas con 1 omitida)
Integración con MongoDB real	9 pruebas sobre un mongod 9.0 desechable: standalone y replica set con transacciones reales
npm run test:e2e (navegador real)	18/18 pasos en Microsoft Edge, sin errores en la consola del navegador
npm audit (backend y frontend)	0 vulnerabilidades
Construcción del frontend	Correcta
Base centro_pediatrico_db	Conservada: 4 pacientes, 3 médicos, 5 vacunas. Solo se agregaron dos índices (no destructivos) y eventos de auditoría de lectura
Validadores de Fase II frente a los datos reales	0 documentos incumplen (verificación de solo lectura)

2. Defectos encontrados en la auditoría y corregidos

La revisión del código, de la base real y de la interfaz en un navegador real encontró defectos que las pruebas anteriores no detectaban:

Defecto	Impacto	Corrección	Evidencia
Auditoría no atómica: si fallaba bitacora_acceso tras una escritura clínica, el evento se perdía	Trazabilidad incompleta	Outbox embebido: el evento viaja en el mismo updateOne y se entrega de forma idempotente	Integración (bitácora caída)
Editar un tratamiento sobrescribía el anterior	Pérdida de información clínica	tratamientos_anteriores con concurrencia optimista	API, integración, interfaz
El médico veía el DPI completo del tutor	Incumple la clasificación de la Fase I	DPI enmascarado para el médico; búsqueda por DPI solo para S y A	API, integración, interfaz, base real
Perímetro cefálico obligatorio (Fase I: "si corresponde")	Inconsistencia con el diseño	Opcional en Zod, interfaz y validador; la API detecta el validador instalado y lo exige mientras no se cambie	API, integración, interfaz
Validadores de Fase II sin las restricciones de Zod	Escrituras directas podían romper el modelo	Longitudes, rangos, patrones e identificadores iguales a Zod	Integración (rechazo de escrituras inválidas)
Sin forma segura de aplicar o revertir validadores	Riesgo operativo	Verificación, respaldo EJSON, collMod y reversión; contraseña pedida de forma oculta	Integración
Scripts mongosh antiguos ejecutables (03 archiva P001; 01 instala un validador incompatible)	Riesgo de alterar datos	Guarda que impide ejecutarlos por accidente	mongosh --nodb

Defecto	Impacto	Corrección	Evidencia
Botones de rol superpuestos en el inicio de sesión	Un clic podía elegir otro rol	Cada opción cubre su propia etiqueta	Interfaz (paso 1)
Tras cerrar sesión, el siguiente usuario volvía a la página del anterior	Navegación y privacidad	Cierre voluntario sin recordar la página	Interfaz (paso 7)
Fechas con hora mostradas en UTC (gráfica 24 oct, tabla 23 oct)	Fechas inconsistentes	Formato local para fechas con hora	Interfaz (paso 9)
Patrones HTML inválidos (el navegador ignoraba la validación)	Validación de cliente inactiva	Patrones escapados	Consola del navegador limpia
Archivar por "otra causa" perdía la categoría	Motivo incompleto	Se guarda la causa y el detalle	Interfaz (paso 15)
Prueba de integración siempre omitida	Adaptador sin prueba real automatizada	mongod desechable propio, independiente de la base principal	9 pruebas

3. Arquitectura hexagonal

```
React --HTTP--> api/ (rutas, Zod, enrutador por institución) --> application/ (casos de uso) --> ports/ <-- adapter
                                                                                                  |                    <-- tests/doubles
                                                                                                  +--> domain/ (reglas de negocio)
```

Capa	Ubicación	Responsabilidad
Dominio	src/domain/	Reglas independientes del motor (tutor principal, fechas, dosis duplicada, archivado, enmascarado de DPI)
Aplicación	src/application/	Casos de uso y auditoría; construye cada evento con id_evento
Puertos	src/ports/	PacienteRepository, MedicoRepository, VacunaRepository, AuditoriaRepository y UnidadDeTrabajo (ejecutar con evento, reenviarPendientes)
Adaptadores	src/adapters/mongo/	Repositorios (updateOne condicionales), unidad de trabajo (transacción u outbox), índices, validadores y su administración
API	src/api/, src/app.js	Rutas, middleware de seguridad y enrutador por institución
Frontend	frontend/src/	Interfaz React que solo usa la API

El mismo conjunto de casos de uso se ejecuta con el adaptador MongoDB (pruebas de integración) y con el doble en memoria (pruebas de API): esto demuestra que la lógica de negocio no depende del motor.

4. Modelo documental

Colección	Tipo	Campos
pacientes	Principal	id_paciente, nombres, apellidos, fecha_nacimiento, sexo, estado_historial, tutores[], consultas[] (diagnósticos, tratamiento, tratamientos_anteriores, prescripciones), controles_crecimiento[] (perímetro opcional), esquema_vacunacion[], trazas, fecha_archivado, motivo_archivado, bitacora_pendiente[] (interna)
medicos	Referenciada	id_medico, colegiado, nombres, apellidos, especialidad{id_especialidad, nombre}
vacunas	Referenciada	id_vacuna, nombre, descripcion
bitacora_acceso	Extensión Fase II	id_evento, id_usuario, rol, id_paciente, accion, recurso, referencia, resultado, fecha_hora, detalle

Dato	Decisión	Justificación
Tutores, consultas, crecimiento, vacunación	Embebido	Se leen con el paciente; cada registro es una sola actualización atómica
Médicos y vacunas	Referenciado	Catálogos reutilizados por muchos pacientes
Bitácora	Colección separada	Crece sin límite; solo se embebe temporalmente el evento pendiente (outbox)

Índice	Propósito
pacientes: id_paciente único · tutores.dpi · estado+apellidos+nombres	Unicidad, búsqueda y listado
pacientes: bitacora_pendiente.id_evento (sparse)	Localizar eventos pendientes
medicos: id_medico único, colegiado único · vacunas: id_vacuna único	Referencias y unicidad
bitacora_acceso: id_paciente+fecha_hora · fecha_hora · id_evento único parcial	Consulta y entrega idempotente

5. Reglas de negocio

- Exactamente un tutor principal; sin id_tutor ni DPI repetidos; paciente duplicado → 409.
- Fechas no futuras (5 min de tolerancia) ni anteriores al nacimiento; receta no anterior a su consulta.
- Cada control de crecimiento pertenece a una consulta existente; las mediciones nunca se sobrescriben.
- Misma vacuna y número de dosis una sola vez (una cancelada permite reprogramar); solo una dosis Pendiente se aplica o cancela.
- Historial archivado: solo lectura; solo el administrador lo consulta. Tratamiento modificado: se conserva la versión anterior.
- Médico y vacuna referenciados deben existir (422). El sistema no genera dosis, calendarios ni recomendaciones.

6. Roles, permisos y seguridad

Operación	Médico	Secretaría	Administrador
Registrar paciente, editar datos y tutores	—	Sí	Sí
Datos generales y tutores	Sí (DPI enmascarado)	Sí	Sí
Historial clínico	Sí (activos)	—	Sí (incluidos archivados)
Consultas, diagnósticos, tratamiento, recetas, crecimiento, vacunación	Lectura y escritura	—	Solo lectura
Archivar, auditoría, archivados, altas de catálogo	—	—	Sí

- **JWT verificado:** se rechazan alg none, tokens sin expiración, emisor o audiencia ajenos, firmas ajenas, tokens alterados (p. ej., un rol cambiado) y tokens vencidos; la institución debe ser centro_pediatrico.
- **Desarrollo local funcional:** /auth/dev-login y scripts/dev-token.js emiten tokens de 8 h y 2 h; se desactivan con NODE_ENV=production.
- **Entradas:** Zod estricto (sin campos desconocidos ni operadores $), parámetros con patrón, consultas repetidas o fuera de rango → 400, JSON malformado → 400, cuerpo mayor a 100 KB → 413.
- **Duplicados:** índices únicos y filtros condicionales (consultas, controles, dosis, diagnósticos, recetas).
- **Auditoría automática:** lecturas, escrituras, accesos denegados e inicios de sesión de desarrollo.
- **Credenciales:** solo en .env (ignorado por Git); la contraseña de administrador para validadores se pide oculta y no se guarda; errores 500 sin detalles internos.

7. Consistencia, transacciones y auditoría

El servidor del proyecto es **standalone**: no admite transacciones multidocumento. Mejoras implementadas sin cambiar su configuración:

- **Atomicidad por documento:** cada operación clínica es un único updateOne condicional (p. ej., aplicar dosis exige estado Pendiente en el filtro). Con dos aplicaciones simultáneas contra MongoDB real, una responde y la otra recibe 409.
- **Outbox embebido:** el evento de auditoría se guarda en el mismo updateOne que el cambio clínico; luego se copia a bitacora_acceso (id_evento único) y se retira. Si la copia falla, se reintenta al arrancar, cada 60 s y al consultar la auditoría. Prueba real: con la bitácora "caída", la consulta se guardó con su evento pendiente, se entregó una sola vez y no se duplicó.
- **Replica set:** la unidad de trabajo usa withTransaction. Prueba real en un replica set desechable: si la bitácora falla, la transacción también revierte la consulta.

Lo que no se afirma: la auditoría no es transaccional en el servidor actual (la copia a bitacora_acceso puede retrasarse); las lecturas, los accesos denegados y las altas de catálogo no usan outbox. Migrar a replica set cambia la configuración global del servidor y requiere autorización (plan con respaldo, riesgos y reversión en docs/OPERACION_MONGODB.md).

8. Validadores $jsonSchema

Colección	Instalado hoy	Fase II (listo)	Documentos que incumplen
pacientes	Versión inicial (perímetro obligatorio)	Reglas iguales a Zod, perímetro opcional	0 de 4
medicos	Ninguno	Sí	0 de 3
vacunas	Ninguno	Sí	0 de 5
bitacora_acceso	Ninguno	Sí	0 de 48

Comando para el administrador (pide la contraseña sin mostrarla; respalda, verifica, aplica e indica cómo revertir):

```
npm run db:validadores -- --aplicar --usuario <usuario_administrador>
npm run db:validadores -- --revertir "respaldos\validadores-<fecha>" --usuario <usuario_administrador>
```

Todo el ciclo (respaldo, aplicación, reversión exacta, documentos idénticos) se comprobó en MongoDB real con una copia del validador instalado hoy.

9. Pruebas ejecutadas

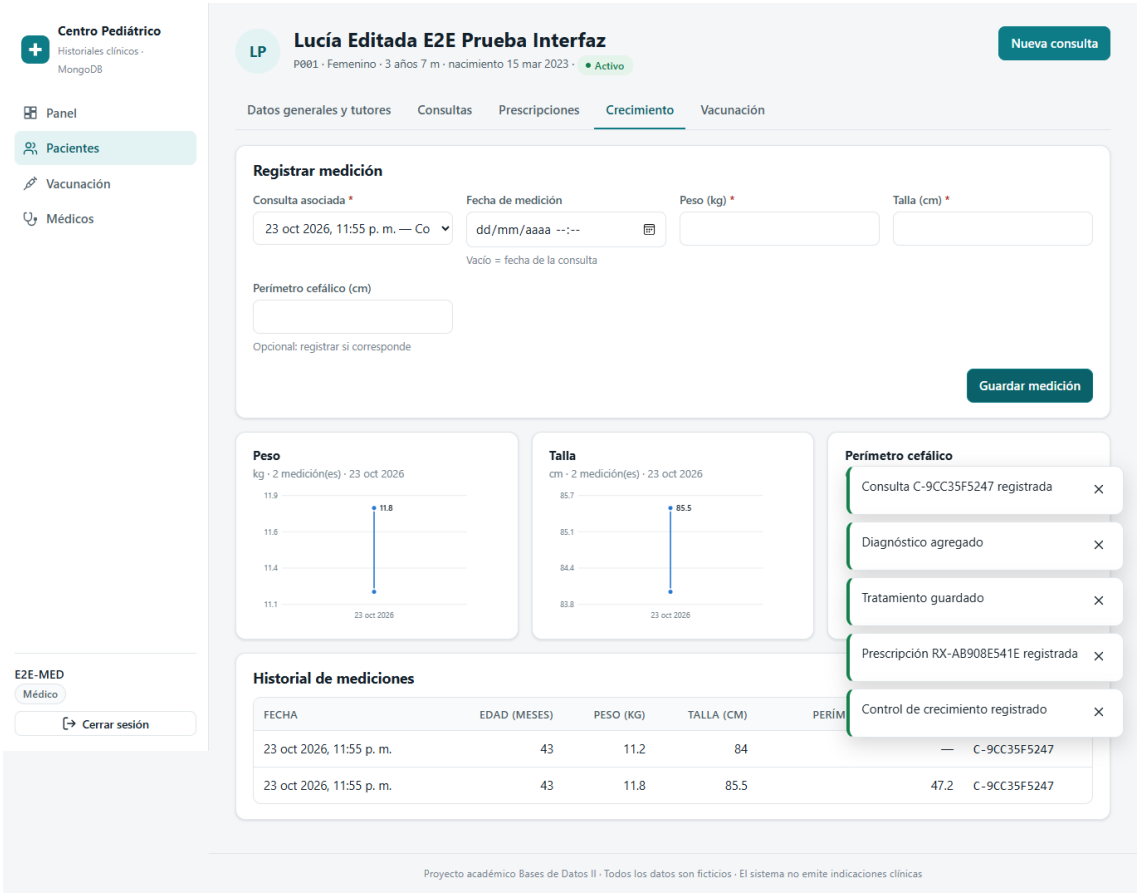
```
npm test            -> tests 61 | pass 61 | fail 0 | skipped 0
npm run test:e2e    -> 18/18 pasos correctos (Microsoft Edge 155, headless)
npm audit           -> found 0 vulnerabilities
npm --prefix frontend audit -> found 0 vulnerabilities
npm run build       -> built correctamente
npm run db:validadores -> 0 documentos incumplen (solo lectura sobre la base real)
```

Archivo	Cobertura
tests/api.test.js	Pacientes, tutores, consultas, diagnósticos, recetas, crecimiento, vacunación (concurrencia), archivado, permisos, auditoría
tests/seguridad.test.js	JWT, DPI, rutas clínicas, parámetros malformados, perímetro, historial de tratamientos, eventos pendientes
tests/enrutamiento.test.js	Institución → módulo/motor; rango de fechas
tests/mongo.integration.test.js	MongoDB real: validadores, outbox, flujo completo, datos heredados, archivado, transacciones
tests/dominio.test.js · schema.test.js	Reglas, puertos y esquemas
tests/frontend-cliente.test.js	Comunicación frontend–backend
tests/e2e/flujo-ui.e2e.js	Interfaz en navegador real (18 pasos)

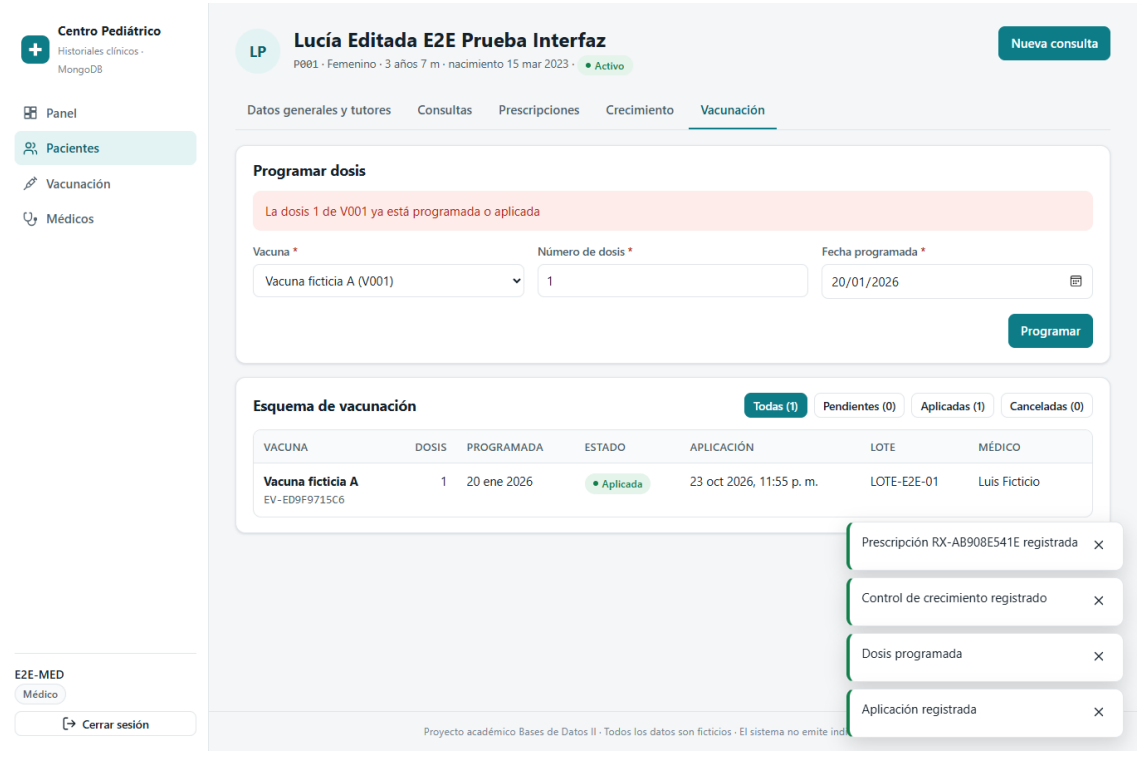
Ninguna prueba usa centro_pediatrico_db: las de integración y de interfaz inician un mongod propio en un puerto libre y una carpeta temporal, y lo eliminan al terminar.

10. Frontend verificado en navegador real

- Inicio de sesión con roles de desarrollo · panel con estadísticas reales · listado vacío y búsqueda sin resultados.
- Registro de un paciente ficticio con dos tutores (incluida la validación del formulario) · expediente de secretaría sin datos clínicos · edición de datos · duplicado rechazado.
- Consulta con diagnóstico, tratamiento, receta y control sin perímetro · diagnóstico, tratamiento (con versión anterior visible) y receta adicionales.
- Segunda medición · tres gráficas y tabla con fechas coincidentes · programación, duplicado rechazado, aplicación con confirmación y doble aplicación rechazada por el servidor (409).
- Historial de medicamentos con filtro · archivo por otra causa · auditoría del expediente, global y de archivados · médico sin acceso al archivado ni a la auditoría.
- Persistencia comprobada en MongoDB (fechas Date, versión anterior del tratamiento, control sin perímetro, sin eventos pendientes).
- Médicos por especialidad · vacunación global · diseño móvil de 390 px sin desplazamiento horizontal y menú funcional · aviso de API caída.



Captura de la prueba de interfaz: pestaña Crecimiento (control sin perímetro y control con perímetro).



Captura de la prueba de interfaz: duplicado rechazado y dosis aplicada (lote y médico).

11. Ejecución en Windows y demostración

```
cd "C:\Users\abela\OneDrive\Pictures\BASES DE DATOS PROYECTO\centro_pediatrico_fase2"
npm install
npm run web:install
npm run build
npm start            # http://localhost:3001

npm test
npm run test:e2e
```

Guion completo en docs/DEMOSTRACION.md: secretaría registra y edita; médico ve el DPI enmascarado, registra la consulta, diagnóstico, tratamiento con versiones, receta, crecimiento y vacunación (doble aplicación rechazada); el administrador archiva y revisa la auditoría; el médico ya no accede al archivado; finalmente se muestran npm test, test:e2e y db:validadores.

A. Funcionalidades verificadas (resumen)

Todas las operaciones del criterio de cierre se comprobaron en navegador real con MongoDB real (desechable): iniciar la aplicación, entrar con los tres roles, usar todas las pantallas, registrar y administrar pacientes, consultas, diagnósticos, tratamientos y prescripciones, crecimiento y vacunación, archivar, permisos y auditoría, y persistencia. Contra la base real se verificó con lecturas: arranque, sesión y capacidades, DPI enmascarado, búsqueda por DPI según el rol, crecimiento, panel y frontend.

C. Limitaciones propias de MongoDB

- **Sin transacciones en el servidor real** (standalone): auditoría por outbox, que no pierde eventos pero no es transaccional. Migrar a replica set requiere autorización; plan listo y la ruta transaccional está probada.
- **Validadores de Fase II sin aplicar** en centro_pediatrico_db por falta de privilegio collMod: hasta aplicarlos, el perímetro cefálico sigue siendo obligatorio y tres colecciones no tienen validador en el servidor.
- Unicidad dentro de arreglos garantizada por filtros condicionales, no por índices. Límite de 16 MiB por documento (suficiente para el volumen pediátrico).
- Lecturas, accesos denegados y altas de catálogo se auditan sin outbox.

D. Pendientes de integración con el equipo

Fuera de la responsabilidad individual:

- Inicio de sesión común con usuarios reales que emita el JWT (contrato en docs/INTEGRACION.md).
- Registrar las otras tres instituciones en el enrutador con sus adaptadores (PostgreSQL, MySQL, Firebase).
- Consultas equivalentes en los cuatro motores, interfaz común y documento comparativo conjunto.
- Resolver la asignación de motores (los documentos del equipo muestran MySQL repetido) y el manejo del secreto JWT entre servicios.